PROJECT STATUS / 26 OCTOBER 2026

Network Intrusion Detection System

Project report and current runtime summary

20 IMPLEMENTATION STEPS COMPLETE | LOCAL APP RUNNING | REAL-DATA MODEL EVIDENCE NOT YET AVAILABLE

The project is a local network-traffic analysis application. It reads an authorized PCAP/PCAPNG capture, groups packets into network flows, calculates versioned features, applies explainable detection rules, optionally runs a verified machine-learning model, saves results to PostgreSQL, and presents persisted results in an authenticated dashboard.

The 20 planned implementation steps are complete and pushed to GitHub main. The local Docker Compose application has now been started. PostgreSQL, the API, and the dashboard reported healthy at startup. The browser is currently on the Alerts page at http://localhost:8080/alerts.

This is a complete software pipeline, but it has not been evaluated on a supplied real dataset. No trained project model is included. Synthetic test results confirm software behavior only; they do not demonstrate real-world detection quality or classifier accuracy.

What the application does

- Reads packet metadata from an authorized offline PCAP or PCAPNG capture. Payloads are not emitted in replay output.
- Groups packets into bidirectional flows and calculates stable, versioned features.
- Applies explainable rules, including TCP SYN-burst and high packet-rate checks.
- Optionally loads a trusted model only when its metadata, checksum, feature schema, and runtime compatibility checks pass.
- Saves capture runs, flows, predictions, alerts, and audit events to PostgreSQL.
- Serves authenticated APIs and a role-based dashboard with overview, alerts, flows, capture history, model status, and administrator account management.
- Streams persisted alerts to authenticated clients over WebSockets.

The dashboard does not capture network traffic by itself. Leaving the Alerts page open does not start monitoring; results appear after a capture is replayed through the monitor, or after an explicitly authorized live capture is configured and started.

Current local runtime

Item	Current state
Compose project	nids-local
PostgreSQL / API / dashboard	Reported healthy at startup; database migrations run with API startup.
Dashboard	http://localhost:8080 (loopback port 8080)
Browser page	/alerts
Local administrator	admin account provisioned after confirming no administrator existed.
Account password	Shared in this conversation; intentionally omitted from project files.
Database volume	Preserved; no volume-deleting command was run.
New capture replayed during startup	None.

The earlier Step 20 synthetic smoke replay used an isolated disposable database. Its generated PCAP and disposable database were removed after the check. That test did not seed the persistent nids-local database, and any older data in that volume was left in place.

System components

Part	Responsibility
nids_engine	Packet replay, flow construction, feature extraction, rules, optional model inference, and monitor CLI.
ml	Feature export, dataset provenance and validation, training, and held-out evaluation.
backend / database	Authentication, APIs, database models, migrations, persistence, audit, and WebSockets.
frontend	React dashboard, login, overview, alerts, flows, capture history, model status, and admin user management.
scripts	Deterministic synthetic demo-capture generator and isolated replay benchmark.
Compose / Docker	Local PostgreSQL, API, one-shot monitor, and dashboard services.

Dashboard routes: /overview, /history, /alerts, /flows, /model, and /users. The Users page is restricted to administrators.

Verification and results

- Backend suite: 74 passed, 2 skipped. The skipped tests are opt-in PostgreSQL integrations; Step 18 PostgreSQL integration coverage was run separately and 2 passed.
- Frontend suite: 15 passed across 3 files.
- TypeScript and production dashboard build passed. Initial JavaScript bundle: 267.48 kB (84.75 kB gzip); chart bundle: 337.60 kB (98.80 kB gzip).
- API, monitor, and dashboard container images built successfully.
- Disposable synthetic demo: 192 packets, 8 flows, and 8 heuristic alerts persisted in its isolated database.

Synthetic replay benchmark

Measure	Result
Input	2,400 synthetic packets (100 flows x 24 packets)
Runs	5 fresh monitor CLI runs, no database or model
Median wall time	23.765572 seconds
Sample range	17.579561 to 25.794867 seconds
Median throughput	100.99 packets/second
Derived time per packet	9.9023 ms

These are machine-specific timings on generated packets. They include process startup and output work. They are not live capture capacity, tail latency, model performance, or proof that real attacks are detected.

Evidence and work still needed

- A legally usable, provenance-tracked labeled dataset whose features match this project's schema.
- Independent capture groups for a valid held-out evaluation and real research metrics.
- A trained and verified model artifact before classifier results can be shown.
- Supported packet-capture drivers and an explicitly authorized interface before live capture can be evaluated.
- Security review and TLS before any public deployment. Login throttling is process-local; distributed rate limiting and access-token revocation lists are not implemented.

Until this evidence exists, describe the result as a completed detection software pipeline with explainable rules and optional model support. Do not report synthetic results as detection accuracy.

GitHub status

The final Step 20 implementation checkpoint is 388ae15 (feat(demo): add reproducible benchmark and final handoff). Steps 18 and 19 were separately pushed as d67a705 and 13d1326. The report and context documents were added afterward in a separate documentation checkpoint.

Operating notes

- The local dashboard is at http://localhost:8080. The generated local admin credential was shared in the conversation; do not copy it into Git or public documentation.
- For an authorized capture, follow docs/DEPLOYMENT.md and run the one-shot Compose monitor service.
- For synthetic demo steps and benchmark caveats, see docs/HANDOFF.md and docs/TESTING.md.
- docker compose down stops services while preserving the database volume. Do not use docker compose down --volumes unless you intend to erase the local database.

Project files

PROJECT_CONTEXT.md provides a short continuation guide. docs/PROJECT_REPORT.md is the editable report source. docs/HANDOFF.md, docs/PROGRESS.md, docs/DEPLOYMENT.md, and docs/KNOWN_LIMITATIONS.md contain the detailed engineering evidence and operating guidance.